

2026년 개인주제탐구발표대회 보고서

AI와 머신러닝을 활용한 구조강아지의 미래예측 프로그램

2026. 9. 4.

과 정 명	AI
소속 학교	탄방중학교
성 명	김 도 윤

대전교육정보원정보영재교육원

I. 탐구배경 및 목적

1. 탐구배경

가. 2026년 4월, 대전 오월드에서 늑대 ‘늑구’가 탈출했다가 약 10일 만에 무사히 구조되었습니다. 당시 많은 시민들은 늑구가 사살되지 않고 안전하게 생포되기를 바랐고, SNS에서도 무사 귀환을 응원하는 목소리가 이어졌습니다. 이를 보며 과거보다 동물의 생명과 복지를 중요하게 생각하는 사회적 인식이 커졌다는 점을 느낄 수 있었습니다.

* (서울신문, 2026. 4.18) ‘늑구야 돌아와서 고마워’

(국제뉴스, 2026. 4.17) ‘대전늑대 늑구, 9일만에 포획, 무사해서 고마워’

나. 농림축산식품부가 2026년 2월 발표한 공식 조사에서도 반려동물 양육 가구 29.2%, 그중 강아지의 비율은 80.5%로 확인되었습니다.

* (연합뉴스, 2026. 2.12) ‘국민 10명중 3명 반려동물 키워, 월평균 양육비 12만원’

다. 하지만 시골에서는 여전히 들개로 인한 피해가 발생하고 있으며, 유기동물 입양 앱 ‘포인핸드’의 자료를 살펴보면 구조된 동물 중 상당수가 자연사하거나 안락사로 생을 마감하고 있는 것으로 나옵니다. 그래서 구조동물, 특히 강아지들의 ① 입양률과 보호자 반환율에 영향을 주는 요인을 알아보고, ② 구조된 강아지의 미래를 예측해 보는 모델을 개발해서, ③ 보다 많은 강아지들의 생존을 위해 무엇을 할 수 있는지 고민보고 싶었습니다.

* (세계일보, 2026. 5.25) ‘사람이 버린 땡땡이가 들개로, 서울 도심서 잇단 입질피해’

(국제신문, 2025. 6.27) ‘유기동물 절반 구조돼도 자연사 혹은 안락사’

2. 탐구목적

가. 구조강아지의 입양률과 보호자반환율에 영향을 주는 요인 통계분석

나. AI와 머신러닝을 통한 구조강아지의 미래예측 프로그램 개발

다. 보다 많은 강아지들의 생존을 위한 제언

※ 전체 구조동물이 아닌 강아지를 분석대상으로 한정 한 이유

- 3년간(2023년~2025년) 구조동물 311,128마리중 강아지 비율이 72%

(강아지 223,944마리, 고양이 82,518마리, 새 등 기타 4,666마리)

- 무게(크기)차이로 동시 비교가 어려움(강아지~80kg, 고양이~8kg)

II. 탐구방법 및 개발환경

가. 공공데이터포털 ‘국가동물보호정보시스템 구조동물 조회서비스’ API를 활용하여 3개년(2023년~2025년) 누적 구조동물 데이터를 활용

농축수산 | 농림축산식품부 | 농림축산검역본부

활용신청 [승인] 농림축산식품부 농림축산검역본부_구조동물 통계 조회 서비스 >

계정 개발 | 신청일 2026-04-15 | 만료예정일 2028-04-15

나. API를 통해 다운로드 받은 원본 csv파일을 구글코랩으로 파이썬 데이터 전처리와 데이터 분석, Chat-GPT Plus Work환경에서 코드 생성

다. 파이썬을 통한 머신러닝 및 최적모델 선정

라. Streamlit의 웹페이지 형태로 구현

III. 데이터 전처리와 분석

1. 데이터 전처리 (전처리 단계별 csv파일로 별도 저장하여 백업)

가. 라이브러리 импорт : request(API에서 자료수집), pandas・numpy(데이터 정리와 분석, re(문자열 입력 데이터 정리), calendar(날짜처리)

나. csv에서 ‘개’ 만 추출 (전체 311,128마리 중 강아지 223,944마리)

```
df_dog = df[df["upKindNm"] == "개"].copy()
```

```
print("전체 데이터:", len(df))
```

```
print("개 데이터:", len(df_dog))
```

전체 데이터: 311128

개 데이터: 223944

다. 나이계산 및 그룹화

```
# 공고시작일 연도 추출
df['notice_year'] = pd.to_datetime(
    df['noticeSdt'].astype(str),
    errors='coerce'
).dt.year

# age_year 숫자형 변환
df['age_year'] = pd.to_numeric(df['age_year'], errors='coerce')

# 공고연도 - 출생연도 = 나이
df['age_num'] = df['notice_year'] - df['age_year']
```

```
# 나이 그룹 구분
df['age_group'] = np.select(
    [
        df['under_60days'].eq('Y') | (df['age_num'] < 1),
        df['age_num'].between(1, 2),
        df['age_num'].between(3, 7),
        df['age_num'] >= 8
    ],
    [
        '1살 미만',
        '1살~2살',
        '3살~7살',
        '8살 이상'
    ],
    default=''
)
```

라. 무게정리 및 그룹화

```
# weight에서 숫자만 추출 → 소수점 첫째자리까지 정리
df['weight_num'] = (
    df['weight']
    .astype(str)
    .str.replace('.', '', regex=False)
    .str.extract(r'(\d+\.\d*)')[0]
    .astype(float)
    .round(1)
)
```

```
df['weight_size_group'] = np.select(
    [
        df['weight_num'] < 5,
        (df['weight_num'] >= 5) & (df['weight_num'] < 10),
        (df['weight_num'] >= 10) & (df['weight_num'] < 15),
        df['weight_num'] >= 15
    ],
    [
        '소형',
        '중소형',
        '중형',
        '대형'
    ],
    default=''
)
```

마. 성별 및 중성화

```
df['sex_group'] = df['sexCd'].map({
    'M': '수컷',
    'F': '암컷'
})
```

```
df['neuter_group'] = df['neuterYn'].map({
    'Y': '중성화',
    'N': '비중성화',
    'U': '미상'
})
```

바. 믹스견과 품종견 구분

```
breed_rank = {breed: rank for rank, breed in enumerate(breed_counts.index, start=1)}

df['breed_rank'] = df['kindNm'].map(breed_rank)
df['breed_count'] = df['kindNm'].map(breed_counts)

rare_breeds = breed_counts.tail(10).index

df['breed_group'] = np.select(
    [
        df['breed_rank'] == 1,
        df['breed_rank'].between(2, 11),
        df['kindNm'].isin(rare_breeds)
    ],
    [
        '믹스견',
        '흔한 품종견',
        '드문 품종견'
    ],
    default='흔하거나 드물지 않은 품종견'
)
```

사. 시지역과 도지역으로 구분

```
df['region'] = df['orgNm'].astype(str).str.extract(
    r'(서울|인천|대전|대구|부산|울산|광주|세종|경기|강원|충북|충남|전북|전남|경북|경남|제주)'
)

df['region_group'] = np.where(
    df['region'].isin(city_regions),
    '시지역',
    '도지역'
)
```

아. 강아지의 결과(입양, 자연사, 안락사, 반환 등) 구분

```
df['outcome_group'] = np.select(
    [
        df['processState'].astype(str).str.contains('입양', na=False),
        df['processState'].astype(str).str.contains('자연사', na=False),
        df['processState'].astype(str).str.contains('안락사', na=False),
        df['processState'].astype(str).str.contains('반환', na=False),
        df['processState'].astype(str).str.contains('보호중', na=False),
        df['processState'].astype(str).str.contains('기증|방사', na=False)
    ],
    [
        '입양',
        '자연사',
        '안락사',
        '반환',
        '보호중',
        '기타'
    ],
    default=''
)
```

자. 불필요한 자료 제거 (현재 보호중이거나 기타 상태 19,740마리, 무게 80kg이상, 나이20살 이상 등 2,282마리 제외, 최종 201,922마리 분석)

```
df = df[~df['outcome_group'].isin(['기타', '보호중'])].copy()

df = df[
    ((df['weight_num'].isna()) | (df['weight_num'] <= 80)) &
    ((df['age_num'].isna()) | (df['age_num'] <= 20))
].copy()
```

2. 데이터 분석결과

가. 강아지의 구조후 결과 : 46.1%는 주인을 찾고, 53.9%는 죽음을 맞이함

구 분	입 양	보호자반환	안락사	자연사	합 계
결 과	58,304마리 (28.9%)	34,813마리 (17.2%)	65,184마리 (32.3%)	43,621마리 (21.6%)	201,922 (100%)

나. 나 이 (단위 : 마리)

구 분	입 양	보호자반환	안락사	자연사	합 계
1살 미만	30,909	2,732	25,969	28,705	88,315
1살~2살	12,457	8,852	17,973	6,107	45,389
3살~7살	13,258	17,759	19,024	7,703	57,114
8살 이상	1,680	5,470	2,218	1,736	11,104
합 계	58,304	34,813	65,184	43,621	201,922

다. 무 게 (단위 : 마리)

구 분	입 양	보호자반환	안락사	자연사	합 계
5kg 미만	37,233	10,218	23,295	29,868	100,614
5~10kg	12,211	10,631	19,305	7,752	49,899
10~15kg	3,795	4,383	10,470	2,869	21,517
15kg 이상	5,065	9,581	12,114	3,132	29,892
합 계	58,304	34,813	65,184	43,621	201,922

라. 견 종 (단위 : 마리)

구 분	입 양	보호자반환	안락사	자연사	합 계
믹 스 견	41,623	13,823	60,869	40,024	156,339
품 종 견	16,681	20,990	4,315	3,597	45,583
합 계	58,304	34,813	65,184	43,621	201,922

마. 성 별

(단위 : 마리)

구 분	입 양	보호자반환	안락사	자연사	합 계
수 컷	29,395	20,381	31,286	21,685	102,747
암 컷	28,909	14,432	33,898	21,936	99,175
합 계	58,304	34,813	65,184	43,621	201,922

바. 중성화

(단위 : 마리)

구 분	입 양	보호자반환	안락사	자연사	합 계
중성화 아님	40,403	17,396	46,607	33,665	138,071
미 상	13,884	11,585	17,309	9,267	52,045
중 성 화	4,017	5,832	1,268	689	11,806
합 계	58,304	34,813	65,184	43,621	201,922

사. 지 역

(단위 : 마리)

구 분	입 양	보호자반환	안락사	자연사	합 계
시 지 역	9,747	10,933	6,565	6,489	33,734
도 지 역	48,557	23,880	58,619	37,132	168,188
합 계	58,304	34,813	65,184	43,621	201,922

아. 데이터 분석 결과

- 입양은 어리고, 작고, 품종견, 중성화 경우가 유리, 성별과 지역은 큰 차이 없음
- 보호자반환은 나이가 있고, 크고, 품종견, 중성화, 시지역인 경우가 유리
- 크기가 있는 시골의 믹스견이 가장 안락사 가능성이 높음

IV. 머신러닝

가. 전처리가 완료된 csv파일을 코랩에 업로드, 전체 자료의 80%는 훈련모델에 사용하고, 20%는 최종평가에 활용

나. 로지스틱회귀, 의사결정나무, 랜덤포레스트 모델 생성 및 학습과 비교를 통한 최적모델 선정

```
models = {
    "로지스틱 회귀": LogisticRegression(
        max_iter=1000,
        random_state=42
    ),
    "의사결정나무": DecisionTreeClassifier(
        max_depth=8,
        min_samples_leaf=100,
        random_state=42
    ),
    "랜덤 포레스트": RandomForestClassifier(
        n_estimators=200,
        min_samples_leaf=50,
        random_state=42,
        n_jobs=-1
    )
}

results = []
trained_models = {}

for model_name, model in models.items():
    pipeline = Pipeline([
        ("preprocessor", preprocessor),
        ("model", model)
    ])

    pipeline.fit(X_train, y_train)

    predicted = pipeline.predict(X_test)
    predicted_probability = pipeline.predict_proba(X_test)

    accuracy = accuracy_score(y_test, predicted)
    macro_f1 = f1_score(y_test, predicted, average="macro")
    probability_loss = log_loss(
        y_test,
        predicted_probability,
        labels=pipeline.classes_
    )

    results.append({
        "모델": model_name,
        "정확도": accuracy,
        "Macro F1": macro_f1,
        "Log Loss": probability_loss
    })

    trained_models[model_name] = pipeline

results_df = pd.DataFrame(results)
results_df
```

다. 랜덤 포레스트는 전체 시험자료의 46.33%를 정확하게 분류, 세 모델 중 정확도가 가장 높고 Log Loss가 가장 낮아, 최종 결과의 확률을 예측하는 모델로 선정함

- 기존 6개 변수 조합별 최대 정확도가 46.6%였기 때문에 최대 예측력에 거의 도달

모델	정확도	Macro F1	Log Loss
랜덤 포레스트	46.33%	0.420	1.142
의사결정나무	46.32%	0.420	1.144
로지스틱 회귀	45.89%	0.445	1.156

라. pkl파일 형태로 전처리데이터와 랜덤 포레스트 머신러닝 모델 저장

- 입력 : 나이, 크기, 성별, 중성화 여부, 견종, 지역
- 모델 : 랜덤 포레스트
- 출력 : 입양 · 반환 · 자연사 · 안락사 확률
- 평가 : 시험자료 20%로 성능 검증
- 정확도 : 46.33% / Log Loss 가장 낮음

마. 6가지 변수 중 예측 중요도 순위

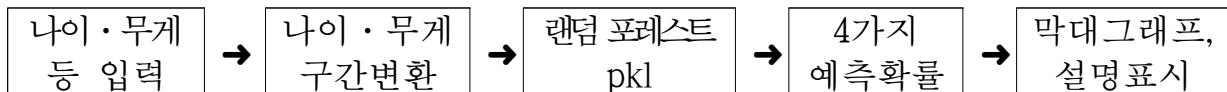
구 분	견 종	크 기	나 이	중성화	지 역	성 별
영향력 비율	42.6%	26.2%	19.9%	5.7%	3.6%	2.1%

V. Streamlit app구현

1. app형태로 구현

가. 랜덤 포레스트 모델이 파이썬의 scikit-learn으로 만들어졌기 때문에 웹 화면 그대로 바꿔주는 Streamlit으로 구현

나. app구조



다. app.py 코딩

```

import streamlit as st
import pandas as pd
import joblib
from pathlib import Path

```

```

# 화면 기본 설정
st.set_page_config(
    page_title="유기견 결과 예측",
    page_icon="🐶",
    layout="centered"
)

```

```

# 저장된 모델 불러오기
@st.cache_resource
def load_model():
    model_path = Path(__file__).parent / "dog_outcome_random_forest.pkl"
    return joblib.load(model_path)

model = load_model()

```

```

# 나이를 모델의 범주로 변환
def convert_age(age):
    if age < 1:
        return "1살 미만"
    elif age <= 2:
        return "1살-2살"
    elif age <= 7:
        return "3살-7살"
    else:
        return "8살 이상"

```

```

# 몸무게를 모델의 범주로 변환
def convert_weight(weight):
    if weight < 5:
        return "소형"
    elif weight < 10:
        return "중소형"
    elif weight < 15:
        return "중형"
    else:
        return "대형"

```

```

# 제목과 안내
st.title("🐶 구조당아지의 미래 예측 프로그램")

st.write(
    "강아지의 정보를 입력하면 기존 유기견 데이터를 학습한 "
    "랜덤 포레스트 모델이 결과별 예측확률을 계산합니다."
)

st.info(
    "이 결과는 실제 강아지의 미래를 확정하는 값이 아니라, "
    "기존 데이터의 통계적 경향을 바탕으로 계산한 모델의 예측확률입니다."
)

```

```
# 입력 화면
st.subheader("강아지 정보 입력")

col1, col2 = st.columns(2)

with col1:
    age = st.number_input(
        "나이(살)",
        min_value=0.0,
        max_value=20.0,
        value=3.0,
        step=0.1
    )

    sex = st.selectbox(
        "성별",
        ["수컷", "암컷"]
    )

    breed = st.selectbox(
        "견종",
        ["믹스견", "품종견"]
    )

with col2:
    weight = st.number_input(
        "몸무게(kg)",
        min_value=0.1,
        max_value=80.0,
        value=7.0,
        step=0.1
    )

    neuter = st.selectbox(
        "중성화 여부",
        ["중성화", "비중성화", "미상"]
    )

    region = st.selectbox(
        "발견 지역",
        ["시지역", "도지역"]
    )

# 예측 버튼
if st.button("예측하기", type="primary", use_container_width=True):
    age_group = convert_age(age)
    weight_group = convert_weight(weight)

    dog_input = pd.DataFrame([
        {"age_group": age_group,
         "weight_size_group": weight_group,
         "sex_group": sex,
         "neuter_group": neuter,
         "breed_group": breed,
         "region_group": region
        }])

    probabilities = model.predict_proba(dog_input)[0]
    result = dict(zip(model.classes_, probabilities))

    display_order = ["입양", "반환", "자연사", "안락사"]
    st.divider()
    st.subheader("예측 결과")

    st.caption(
        f"모델 입력 변수: {age_group} · {weight_group} · "
        f"{sex} · {neuter} · {breed} · {region}"
    )

result_columns = st.columns(4)

for column, outcome in zip(result_columns, display_order):
    with column:
        st.metric(
            label=outcome,
            value=f"{result[outcome] * 100:.1f}%"
        )

st.subheader("결과별 예측확률")

for outcome in display_order:
    probability = float(result[outcome])
    st.write(f"*{outcome}: {probability * 100:.1f}%")
    st.progress(probability)

highest_outcome = max(display_order, key=lambda x: result[x])

st.success(
    f"이 조건에서 가장 높게 예측된 결과는 "
    f"{highest_outcome} 이며, 예측확률은 "
    f"{result[highest_outcome] * 100:.1f}%입니다."
)

st.caption(
    "※ 본 결과는 구조동물의 미래 결과를 단정하거나, "
    "개별 동물의 가치를 판단하기 위한 자료가 아닙니다."
)
```

라. cloudflare연결도구 설치 및 streamlit 실행

마. 프로그램을 로컬PC로 다운로드, zip파일로 생성

바. localhost:8501 작동용 설치.bat 및 실행.bat파일 생성

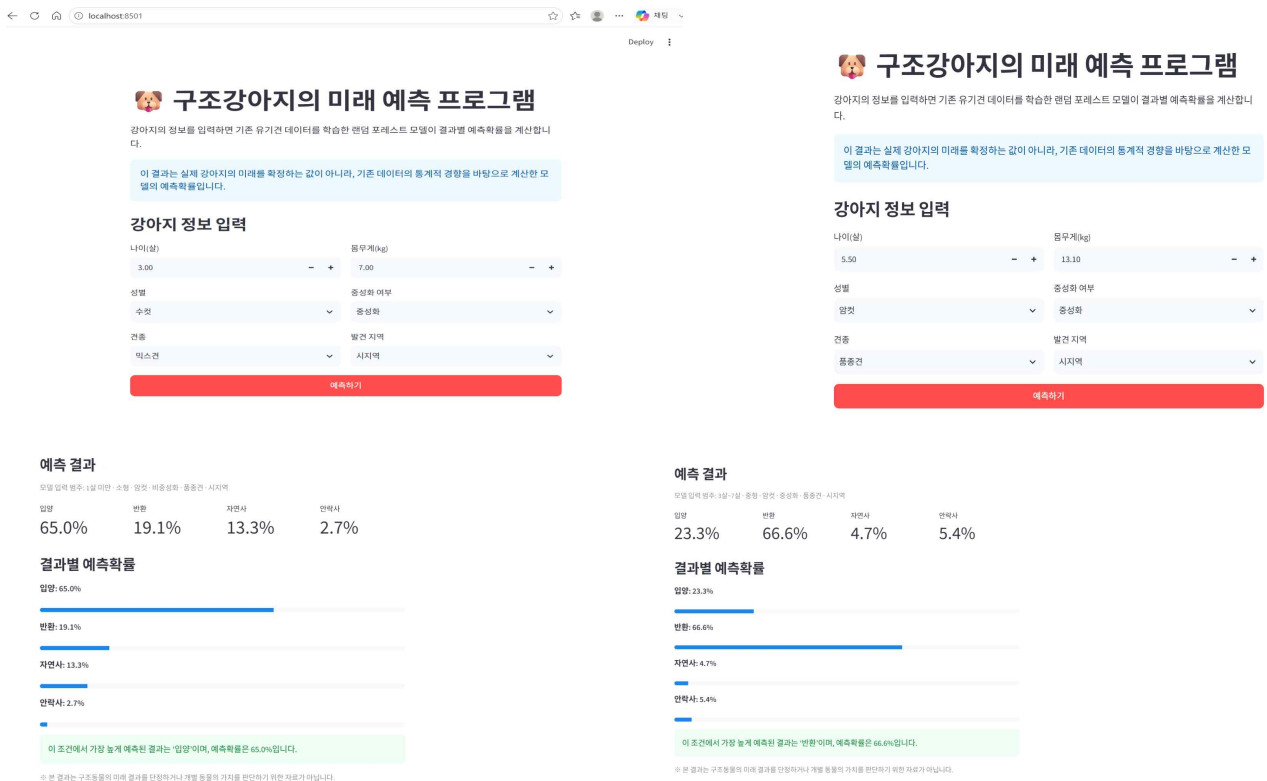
2. 실행

가. 실행환경 : 파이썬 3.12이상이 사전에 local PC에 설치되어 있어야함

나. 실행방법

- 압축파일을 원하는 폴더에 해제
- 설치.bat을 통해서 라이브러리 설치 후 실행.bat으로 실행

다. 실행화면



(예시)1살미만, 소형, 암컷, 비중성화, 품종견, 시지역
→ 입양확률이 가장 높음

(예시)3~7살, 중형, 암컷, 중성화, 품종견, 시지역
→ 보호자 반환 확률이 가장 높음

예측 결과

모형 입력 변수: 나이, 무게, 품종, 품종-중성화, 품종-지역, 품종-지역-칩있음

입양 36.4% 반환 1.1% 자연사 36.2% 안락사 26.3%

결과별 예측확률



이 조건에서 가장 높게 예측된 결과는 '입양'이며, 예측확률은 36.4%입니다.

* 본 결과는 구조동물의 미래 결과를 단정하거나 개별 동물의 가치를 판단하기 위한 자료가 아닙니다.

(예시) 1살미만, 소형, 암컷, 비중성화, 믹스견, 도지역
→ 입양과 자연사 확률이 모두 높음

예측 결과

모형 입력 변수: 나이, 무게, 품종, 품종-중성화, 품종-지역, 품종-지역-칩있음

입양 11.5% 반환 18.5% 자연사 24.8% 안락사 45.2%

결과별 예측확률



이 조건에서 가장 높게 예측된 결과는 '안락사'이며, 예측확률은 45.2%입니다.

* 본 결과는 구조동물의 미래 결과를 단정하거나 개별 동물의 가치를 판단하기 위한 자료가 아닙니다.

(예시) 8살이상, 대형, 암컷, 비중성화, 믹스견, 도지역
→ 안락사 확률이 가장 높음

Ⅶ. 제언 및 느낀점

1. 제언

- 가. 이 프로그램을 활용하여 입양이나 보호자반환 확률이 높은 경우 좀더 장기간 보호하는 것이 많은 강아지의 생명을 지키는데 도움이 될 것 같습니다. (지금은 보호소에서 10일의 공고기간이 끝나면 자의적으로 안락사 결정)

나이 많음, 큰	보호자반환율 높음 나이 많고 크고, 품종견-중성화-시지역-칩있음	안락사 높음 믹스견-비중성화-도지역-칩없음
	입양률 높음 어리고 작고, 품종견-중성화	자연사 높음 어리고 작고, 믹스견
나이, 무게 (크기)	품종-중성화-시지역-칩있는 경향성이 큼 '도시품종개'	믹스-비중성화-도지역-칩없는 경향성이 큼 '시골마당개'

구분	입양	보호자반환	자연사	안락사
나이	어린	많은	어린	-
무게	소형	대형	소형	중형이상
견종	품종견	품종견	믹스견	믹스견
중성화	Y	중성화	비중성화	비중성화
지역	-	시지역	-	도지역

- 나. 나이, 무게, 지역데이터를 보다 세분화 시키고, 강아지 색깔과 같은 추가적인 정보를 반영할 경우 더 예측확률이 높은 모델을 만들 수 있을 것 같습니다.

2. 느낀점

- 가. 데이터를 찾고 전처리 하는 과정이 이후의 머신러닝과 어플로 만드는 것보다 훨씬 많은 시간을 필요로 했습니다.
- 나. GPT와의 대화를 통해 어플구조와 실행코드를 만들 수 있었습니다. 특히, 머신러닝 모델 선정, 어플 플랫폼 선정과정에서 도움이 많았습니다.
- 다. 직접적인 코딩보다는, 무엇을 어떻게 왜 하는지를 계획하고 결정하는 부분이 훨씬 더 중요하였습니다.